

Quest1

Multimodal Dialogue Localization

Dialogue Localization System

Final System Architecture

Contents

1	Current Approach	1
1.1	ASR Pipeline	2
1.1.1	Gap-Based Secondary ASR	3
1.1.2	ASR Target Matching	3
1.2	OCR Pipeline	3
1.2.1	OCR Processing Model	4
1.3	ASR–OCR Integrated Pipeline	4
1.4	Temporal Refinement	4
1.5	Optimizations Applied to the Final Approach	5
1.5.1	ASR Optimizations	5
1.5.2	OCR Optimizations	5
1.5.3	ASR–OCR Optimizations	5
1.6	Models Used in the Final System	6
1.7	Final Pipeline Architecture	7
2	Approaches Attempted	7
2.1	Approach 1: Faster-Whisper Tiny	8
2.1.1	Advantage	8
2.1.2	Limitation	8
2.2	Approach 2: Faster-Whisper Base	8
2.2.1	Advantage	9
2.2.2	Limitation	9
2.3	Approach 3: Faster-Whisper Small	9
2.3.1	Advantage	9
2.3.2	Limitation	9
2.4	Approach 4: Faster-Whisper Medium	10
2.4.1	Advantage	10
2.4.2	Limitation	10
2.5	Approach 5: Hybrid Small + Tiny ASR	10
2.5.1	Step 1: Primary Small Model	11
2.5.2	Step 2: Detect Missing Regions	11
2.5.3	Step 3: Selective Tiny Processing	11
2.5.4	Step 4: Merge the Results	11
2.5.5	Why the Hybrid Approach Was Chosen	11
2.6	Comparison of ASR Approaches	12
2.7	Final Selection Rationale	12
2.8	Final Approach Summary	13
2.9	ASR Model Comparison	13
3	System Limitations	14
3.1	ASR (Automatic Speech Recognition) Limitations	14
3.1.1	Tiny Model (<code>tiny.en</code>)	14
3.1.2	Small Model (<code>small.en</code>)	14
3.1.3	Medium Model (<code>medium.en</code>)	15
3.2	OCR (Optical Character Recognition) Limitations	15
3.3	Video Processing and Frame Extraction	15
3.4	Temporal Refinement Limitations	15
3.5	Fuzzy Matching False Positives	15

4	Optimizations	16
4.1	Implemented Optimizations	16
4.1.1	ASR Audio-Anchoring Hypothesis (Early Halting)	16
4.1.2	Coarse-to-Fine Temporal Binary Search	16
4.1.3	Grayscale and Blank Frame Binarization	16
4.1.4	Persistent Transcript Caching & RAM Caching	16
5	Performance Optimization and Bottleneck Analysis	17
5.1	Component-Level Performance Analysis	17
5.1.1	ASR (Automatic Speech Recognition)	17
5.1.2	Video and OCR Processing	17
5.1.3	Matching & Refinement	17
5.1.4	I/O and Disk Operations	17
5.2	Performance Bottleneck Analysis	17
5.2.1	Primary Bottleneck: Hybrid ASR FFmpeg Cropping	17
5.3	Limitations of the Hybrid Approach	18
5.4	Future Optimization Roadmap	18
5.4.1	High Priority	18
5.4.2	Medium Priority	18
5.4.3	Dynamic Model Loading & VRAM Sharing	18
5.4.4	Low Priority	18

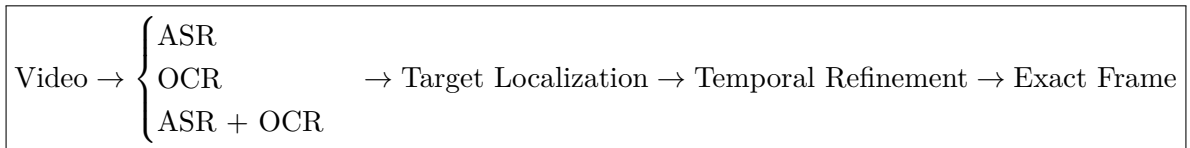
1 Current Approach

The current system uses a multimodal dialogue localization architecture consisting of three complementary pipelines:

1. **ASR Pipeline** – identifies spoken dialogue from the audio.
2. **OCR Pipeline** – identifies visually displayed dialogue or text from video frames.
3. **ASR–OCR Pipeline** – combines audio-based temporal localization with visual verification and temporal refinement.

The final system is designed around the principle of using the least expensive reliable method first and applying additional processing only when required.

The overall architecture is:



1.1 ASR Pipeline

The ASR pipeline is responsible for locating dialogue that is spoken in the video.

The final ASR implementation uses a hybrid combination of two Faster-Whisper models:

`small.en + tiny.en`

The processing flow is:

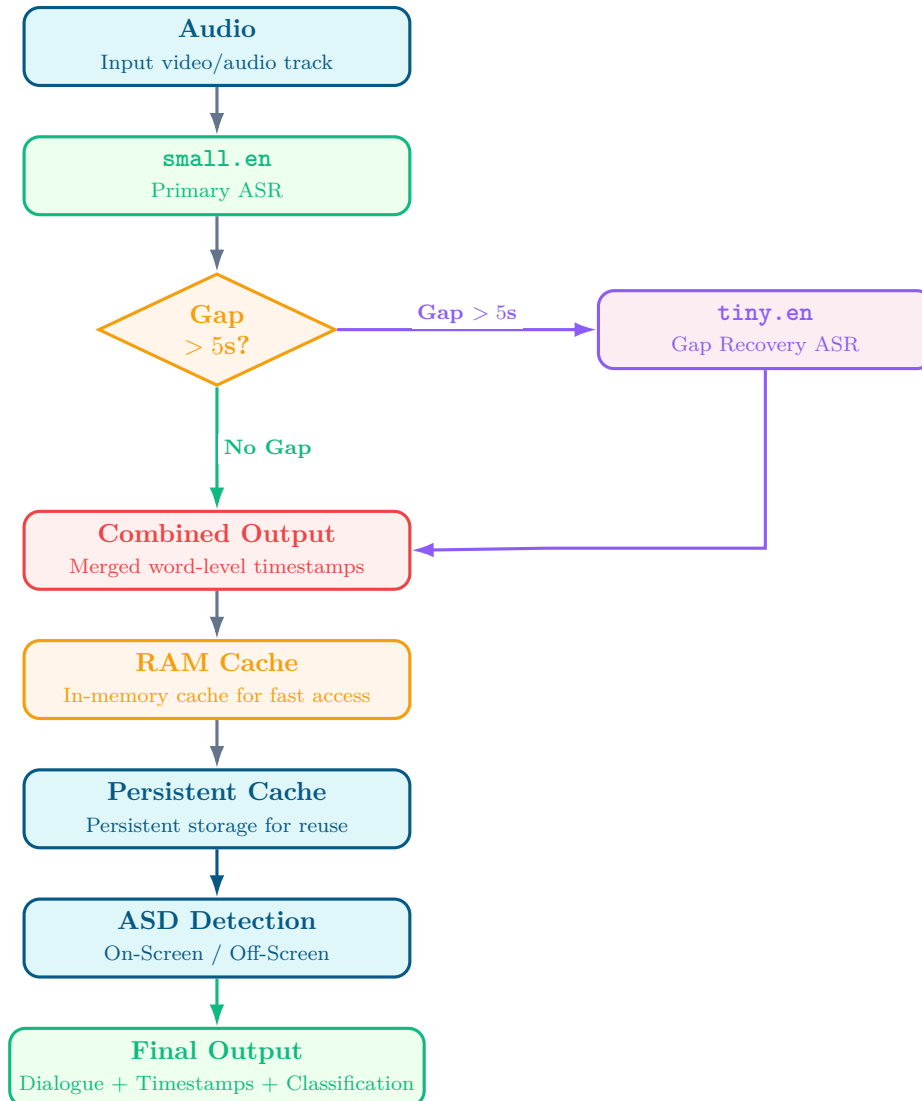


Figure 1: Final hybrid ASR pipeline with caching and Active Speaker Detection. The input audio is first processed using `small.en` as the primary ASR model. The resulting word-level timeline is checked for gaps greater than five seconds. If no such gap is detected, the `small.en` result is passed directly to the combined output. When a gap greater than five seconds is detected, the corresponding audio region is reprocessed using `tiny.en`, and the recovered words are merged with the primary ASR results. The combined word-level output is then cached in RAM for fast repeated access and persisted to storage for reuse across application sessions. The resulting dialogue timestamp is subsequently passed to the Active Speaker Detection (ASD) stage to classify the dialogue as ON-SCREEN or OFF-SCREEN. The final output contains the dialogue, timestamps, and speaker classification.

The `small.en` model acts as the primary transcription engine and generates a word-level transcript containing temporal information.

The primary model was selected to provide a balance between transcription quality and processing speed.

1.1.1 Gap-Based Secondary ASR

After the primary transcription is generated, the word-level timeline is examined for large temporal gaps.

The current trigger is:

$$\Delta t > 5 \text{ seconds}$$

A gap greater than five seconds is treated as a candidate region where the primary ASR may have failed to recognize dialogue.

Only these regions are reprocessed using:

$$\text{tiny.en}$$

Therefore, `tiny.en` is not applied to the complete audio.

The decision process is:

$$\text{small.en} \rightarrow \begin{cases} \Delta t \leq 5s & \rightarrow \text{Keep Primary Result} \\ \Delta t > 5s & \rightarrow \text{Reprocess Region with tiny.en} \end{cases}$$

The resulting word-level information is combined into a single temporal timeline.

1.1.2 ASR Target Matching

The target dialogue is searched within the resulting transcript using a hierarchical matching strategy:

$$\text{Exact} \rightarrow \text{Sequential Partial} \rightarrow \text{Fuzzy}$$

Exact matching is attempted first. If the complete target is not found, sequential partial matching is considered. Fuzzy matching is then used to handle transcription variations.

The ASR pipeline therefore produces:

$$\text{Target Dialogue} \rightarrow \text{Word Match} \rightarrow \text{Timestamp}$$

1.2 OCR Pipeline

The OCR pipeline is responsible for locating dialogue or text that is visually displayed in the video.

The pipeline is:



Figure 2: OCR-only dialogue localization pipeline from video frame sampling to the exact target frame.

The OCR pipeline does not rely on the audio transcript.

It can therefore be used when the target text is visually present but cannot be reliably located through ASR.

The OCR search follows a coarse-to-fine strategy.

Initially, the video is sampled coarsely to identify a candidate temporal region. OCR is then applied to the candidate region and the search interval is progressively reduced until the target frame is identified.



Figure 3: Coarse-to-fine temporal refinement used to progressively narrow the search interval from a broad temporal region to the exact target frame.

1.2.1 OCR Processing Model

The OCR implementation provides the visual text recognition stage of the pipeline.

The OCR model itself is kept independent from the temporal search strategy. The main optimization is to reduce the number of frames and image regions that require OCR inference.

1.3 ASR–OCR Integrated Pipeline

The ASR–OCR pipeline combines the two modalities.

ASR is used to obtain an efficient temporal estimate, while OCR is used to verify or refine the visual location.

The integrated pipeline is:



Figure 4: Final integrated ASR–OCR pipeline. The hybrid ASR stage first identifies the target dialogue and its timestamp. The timestamp is then used to localize the OCR search region, after which coarse-to-fine temporal refinement identifies the exact target frame.

When the target dialogue is successfully identified by ASR, the resulting timestamp is used to restrict the OCR search.

Instead of processing the entire video:

Full Video $\rightarrow$ OCR,

the system performs:

$$t_{\text{ASR}} \rightarrow [t_{\text{ASR}} - \Delta, t_{\text{ASR}} + \Delta] \rightarrow \text{OCR}$$

This substantially reduces the visual search space.

If ASR cannot reliably identify the target, the system can fall back to the OCR-only pipeline.

Thus:

$$\text{ASR} \rightarrow \begin{cases} \text{Reliable Match} & \rightarrow \text{Localized OCR} \\ \text{No Reliable Match} & \rightarrow \text{OCR Fallback} \end{cases}$$

1.4 Temporal Refinement

All three processing paths ultimately use temporal refinement when a precise frame location is required.

The temporal search progresses from a coarse interval to a fine interval:

$$1.0\text{ s} \rightarrow 0.1\text{ s} \rightarrow 0.01\text{ s} \rightarrow \text{Frame Level}$$

This avoids performing expensive frame-level processing over the complete video.

The timestamp is converted to an explicit frame index using:

$$F = \lfloor t \times FPS \rfloor$$

The final stage extracts the corresponding video frame.

1.5 Optimizations Applied to the Final Approach

The final pipelines were optimized independently and at the points where the different modalities interact.

1.5.1 ASR Optimizations

The ASR pipeline uses:

- **Hybrid ASR:** `small.en` is used for the primary transcription and `tiny.en` is used only for large gaps.
- **Selective Reprocessing:** The secondary model is not run over the complete audio.
- **Word-Level Timestamps:** Word-level temporal information is retained for accurate dialogue localization.
- **Hierarchical Matching:** Exact, sequential partial, and fuzzy matching are attempted in order.

1.5.2 OCR Optimizations

The OCR pipeline includes:

- Subtitle-region cropping.
- Blank-frame filtering.
- Resolution scaling.
- Visual frame deduplication.
- Controlled OCR worker concurrency.
- GPU synchronization.

These optimizations reduce unnecessary OCR inference and GPU usage.

1.5.3 ASR–OCR Optimizations

The integrated pipeline uses:

- ASR-guided temporal localization.
- Localized OCR windows.
- OCR fallback only when required.
- Coarse-to-fine temporal refinement.
- Reuse of previously processed ASR/OCR information.

The primary optimization is therefore to avoid expensive visual processing when the audio already provides a reliable temporal location.

1.6 Models Used in the Final System

The final system uses different models according to the processing requirement.

Pipeline	Model	Role
ASR	<code>small.en</code>	Primary full-audio transcription.
ASR	<code>tiny.en</code>	Selective reprocessing of regions with gaps greater than five seconds.
OCR	OCR engine	Recognition of visually displayed text in selected video frames.
ASR-OCR	Hybrid ASR + OCR	Combines spoken-dialogue localization with visual verification and temporal refinement.

Table 1: Models used in the Current pipelines

1.7 Final Pipeline Architecture

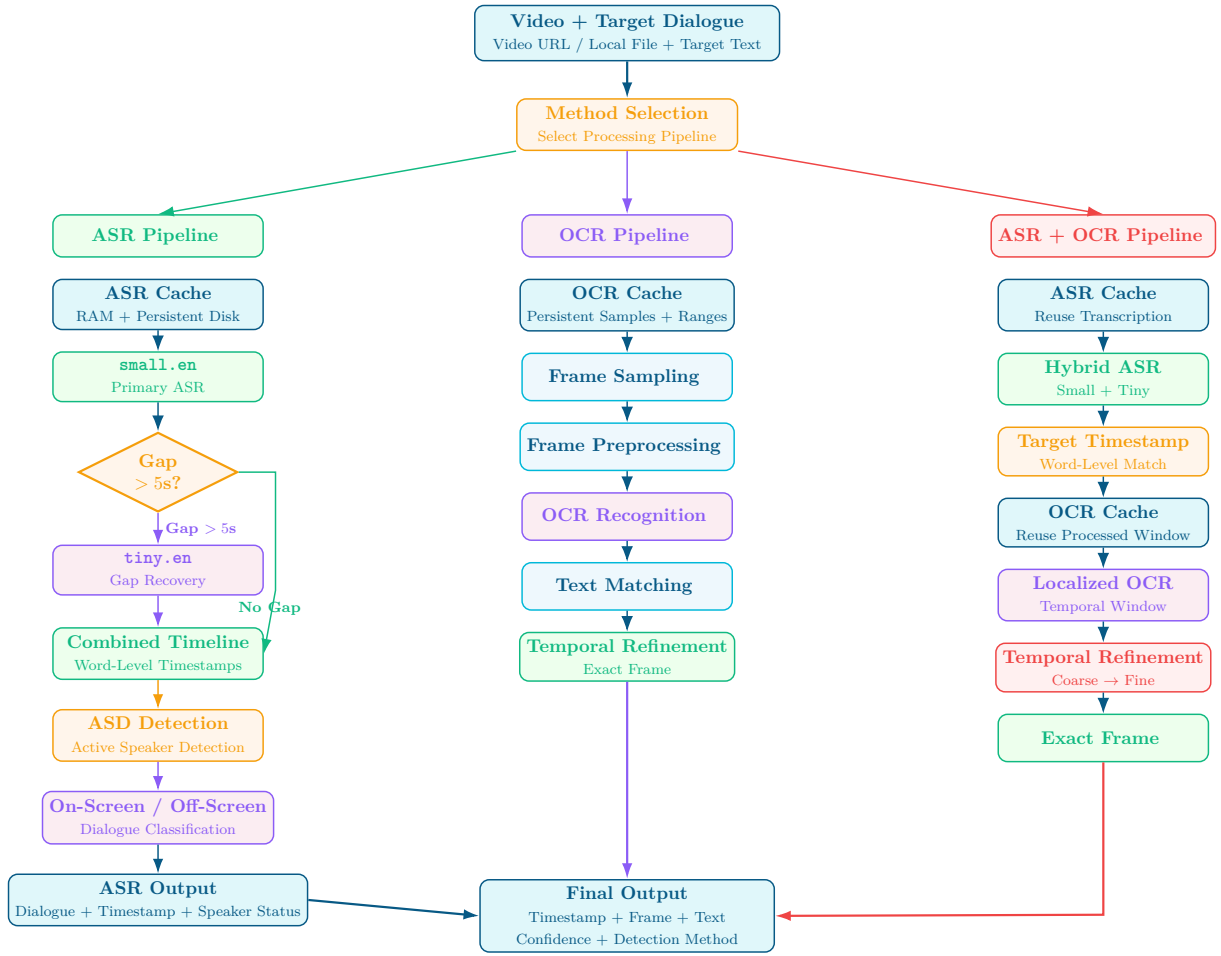


Figure 5: Current multimodal dialogue localization architecture. The system receives a video and target dialogue and selects an ASR, OCR, or integrated ASR–OCR processing pipeline. The ASR pipeline uses a cached transcription when available, otherwise performs primary ASR using `small.en` and selectively reprocesses temporal gaps greater than five seconds using `tiny.en`. The resulting word-level timeline is passed to Active Speaker Detection for on-screen/off-screen dialogue classification. The OCR pipeline similarly checks its persistent cache before performing frame sampling, preprocessing, OCR recognition, text matching, and temporal refinement. The integrated pipeline reuses cached ASR results, obtains the target timestamp, checks the OCR cache for the localized temporal window, and performs localized OCR followed by coarse-to-fine temporal refinement. All three pipelines converge into the final output.

2 Approaches Attempted

The ASR component was developed through a series of model experiments. The main objective was to find a suitable balance between transcription accuracy and processing speed, particularly for dialogue mixed with background music.

The progression of the ASR approaches was:

`tiny.en` → `base.en` → `small.en` → `medium.en` → `small.en` + `tiny.en`

Each approach was selected based on the limitation observed in the previous model.

2.1 Approach 1: Faster-Whisper Tiny

The first ASR approach used the lightweight Faster-Whisper model:

`tiny.en`

The initial ASR pipeline was:

Audio → `tiny.en` → Target Dialogue Matching → Dialogue Timestamp

The main reason for selecting `tiny.en` was its high processing speed and low computational requirement. Since the objective of the ASR stage was to quickly locate the target dialogue within the video, a lightweight model was initially preferred.

2.1.1 Advantage

The primary advantage of `tiny.en` was its fast inference speed.

`tiny.en` → Very Fast ASR

The model was capable of recognizing the spoken content and generating timestamps quickly, making it suitable as an initial ASR solution.

2.1.2 Limitation

The main limitation was not that `tiny.en` failed to detect speech. Instead, the problem was the accuracy of the recognized words.

In some cases, the model produced phonetically similar or incorrect words. For example, a spoken word such as “bare” could be transcribed as “bear” or another similar-sounding word.

Therefore, the problem occurred at the word-recognition level:

Spoken Word → `tiny.en` → Incorrect Similar-Sounding Word

This created a problem for the target dialogue matching stage. Even though the dialogue was present in the audio and the ASR produced a timestamp, the target text could fail to match because the recognized word was different from the expected word.

The resulting trade-off was:

Very Fast + Lower Word-Level Accuracy

Therefore, the next approach was to use a larger ASR model with higher transcription accuracy so that phonetically similar words would be recognized more reliably.

2.2 Approach 2: Faster-Whisper Base

The second approach replaced `tiny.en` with:

`base.en`

The pipeline architecture remained unchanged:

Audio → `base.en` → Target Matching → Timestamp

The objective was to improve the transcription accuracy while maintaining the speed advantage of a relatively small ASR model.

2.2.1 Advantage

Compared with `tiny.en`, the `base.en` model provided better transcription quality while remaining relatively fast.

Therefore, the tradeoff improved to:

Fast + More Accurate

This made `base.en` a better candidate for the primary ASR stage.

2.2.2 Limitation

However, testing showed that `base.en` could still leave words out when the dialogue was mixed with strong background music.

The problem therefore remained:

Background Music → Missed Dialogue Words

Although the model was faster and more accurate than `tiny.en`, it was still not sufficiently robust for difficult audio conditions.

A larger model was therefore tested.

2.3 Approach 3: Faster-Whisper Small

The third approach used:

`small.en`

The pipeline was:

Audio → `small.en` → Target Matching → Timestamp

The objective was to obtain a better balance between speed and accuracy.

2.3.1 Advantage

The `small.en` model provided a significant improvement in transcription quality while still maintaining good processing speed.

Therefore:

`small.en` → Fast + High Accuracy

Compared with the previous models, it provided a stronger baseline for dialogue extraction.

2.3.2 Limitation

Despite the improvement, `small.en` could still miss portions of dialogue when the speech was heavily masked by background music.

Therefore, the problem was no longer that the complete transcript was unreliable. Instead, the failure was concentrated in particular regions.

The observed behavior was:

Most Dialogue → Successfully Transcribed

but:

Difficult Audio Region → Missing Words

This suggested that increasing the model size for the entire video might not be the most efficient solution.

Nevertheless, a larger model was tested to determine whether the problem could be eliminated completely.

2.4 Approach 4: Faster-Whisper Medium

The fourth approach upgraded the ASR model to:

`medium.en`

The pipeline remained:

Audio → `medium.en` → Target Matching → Timestamp

The objective was to maximize transcription robustness and determine whether the background-music problem could be substantially reduced by using a larger ASR model.

2.4.1 Advantage

The `medium.en` model provided stronger transcription robustness than the smaller models.

The larger model therefore reduced the number of dialogue words missed in difficult audio conditions.

The tradeoff became:

Higher Accuracy

2.4.2 Limitation

The major problem was processing speed.

The larger model introduced significantly higher inference cost, making the overall pipeline too slow for the intended application.

Therefore:

`medium.en` → Good Accuracy + Too Slow

This showed that simply selecting the largest model was not an appropriate solution.

The system needed to retain the speed of `small.en` while recovering the words that it missed in difficult regions.

2.5 Approach 5: Hybrid Small + Tiny ASR

The final approach combines the strengths of the previous two useful models:

`small.en` + `tiny.en`

Instead of running a large model over the complete video, the system uses `small.en` as the primary model and selectively uses `tiny.en` to recover potentially missed regions.

The final ASR pipeline is:

Audio → `small.en` → Gap Detection → `tiny.en` → Transcript Integration

2.5.1 Step 1: Primary Small Model

The complete audio is first processed using:

`small.en`

This provides the primary word-level transcript and timestamps.

The reason for using `small.en` as the primary model is its favorable balance between speed and transcription accuracy.

2.5.2 Step 2: Detect Missing Regions

After transcription, the temporal distance between consecutive recognized words is examined.

A large gap indicates a possible region where the primary ASR model failed to recognize speech. The current threshold is:

$\Delta t > 5.0$ seconds

Such regions are treated as candidate missing-dialogue regions.

2.5.3 Step 3: Selective Tiny Processing

Instead of running `tiny.en` over the complete audio, only the identified regions are reprocessed:

Detected Gap $\rightarrow$ Audio Chunk $\rightarrow$ `tiny.en`

The `tiny.en` model is therefore used as a recovery model rather than the primary model.

2.5.4 Step 4: Merge the Results

The words detected by `tiny.en` are timestamp-adjusted according to their original position in the video.

They are then inserted into the primary `small.en` transcript.

The resulting timeline is:

`small.en` + `tiny.en` Recovery = Combined Word-Level Timeline

2.5.5 Why the Hybrid Approach Was Chosen

The previous experiments established the following:

<code>tiny.en</code>	$\rightarrow$	Very Fast, but Less Accurate
<code>base.en</code>	$\rightarrow$	Fast and More Accurate, but Misses Words Under Music
<code>small.en</code>	$\rightarrow$	Fast and Very Accurate, but Still Misses Some Words Under Music
<code>medium.en</code>	$\rightarrow$	More Robust, but Too Slow

The hybrid approach therefore combines the useful properties of `small.en` and `tiny.en`:

Small $\rightarrow$ Primary Accurate + Fast Transcription

Tiny $\rightarrow$ Fast Recovery of Difficult Regions

The final strategy is therefore:

Use Small Everywhere + Use Tiny Only Where Required

This avoids the computational cost of running `medium.en` over the entire audio while still providing an additional mechanism for recovering dialogue missed by `small.en`.

The repository’s final hybrid implementation follows this gap-based strategy: `small.en` performs the primary sweep, temporal gaps greater than 5 seconds are detected, and only those isolated regions are reprocessed with `tiny.en`.

2.6 Comparison of ASR Approaches

Approach	Model	Observation	Limitation
Approach 1	<code>tiny.en</code>	Very fast and lightweight.	Lower accuracy; misses dialogue under difficult background music.
Approach 2	<code>base.en</code>	Fast and more accurate than <code>tiny.en</code> .	Still misses words when dialogue is mixed with background music.
Approach 3	<code>small.en</code>	Fast and very accurate; strong overall balance.	Still misses some dialogue in heavily music-masked regions.
Approach 4	<code>medium.en</code>	Higher robustness for difficult audio.	Processing time is too high for the intended application.
Final	<code>small.en</code> + <code>tiny.en</code>	Uses <code>small.en</code> for the complete audio and <code>tiny.en</code> only for detected gaps.	Additional processing is required for detected gap regions.

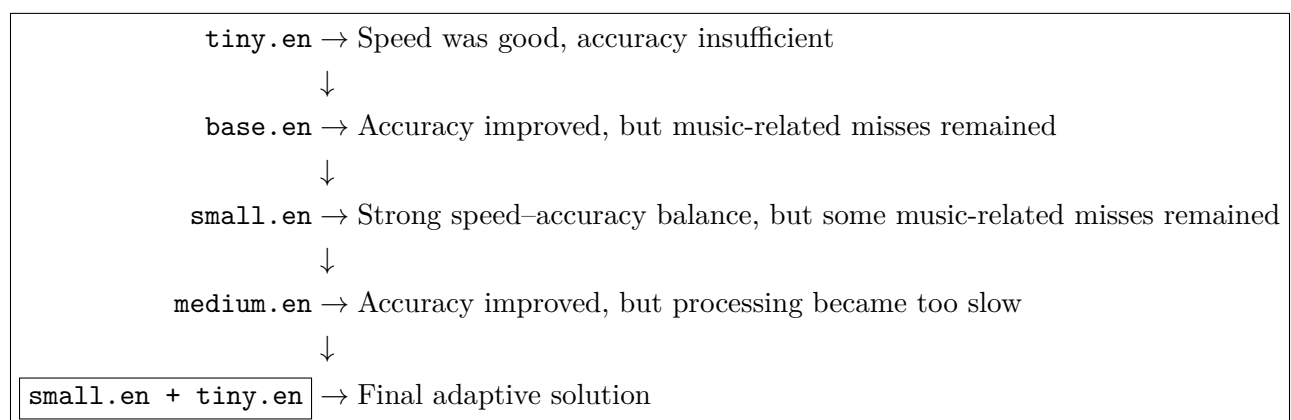
Table 2: Evolution of the ASR models during development

2.7 Final Selection Rationale

The final model was not selected simply because it was the largest or most accurate model.

Instead, it was selected based on the overall speed–accuracy tradeoff.

The progression was:



The final architecture therefore avoids the two extreme choices:

Tiny: Too Little Accuracy

and

Medium: Too Much Processing Time

Instead, the system uses:

Small for Speed + Accuracy

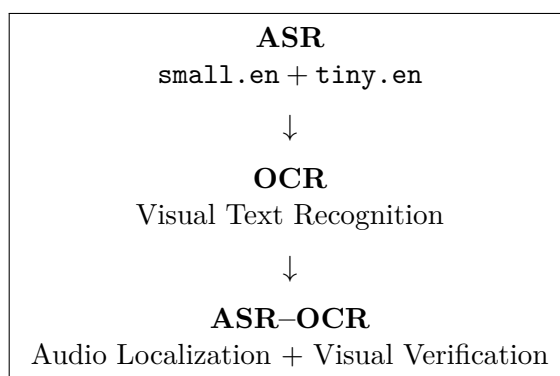
and:

Tiny for Selective Recovery

This provides the final balance required by the dialogue localization system.

2.8 Final Approach Summary

The final approach combines three complementary pipelines:



The key design decision is that `small.en` performs the primary ASR processing, while `tiny.en` is selectively used only for regions with gaps greater than five seconds.

The final system therefore attempts to balance:

Accuracy + Processing Speed + Temporal Precision

while avoiding unnecessary full-video visual processing whenever reliable audio information is available.

2.9 ASR Model Comparison

The following graph provides an illustrative comparison of the four standalone Faster-Whisper models and the final hybrid approach evaluated during development. The values are illustrative and are used only to visualize the observed speed-accuracy trade-off.

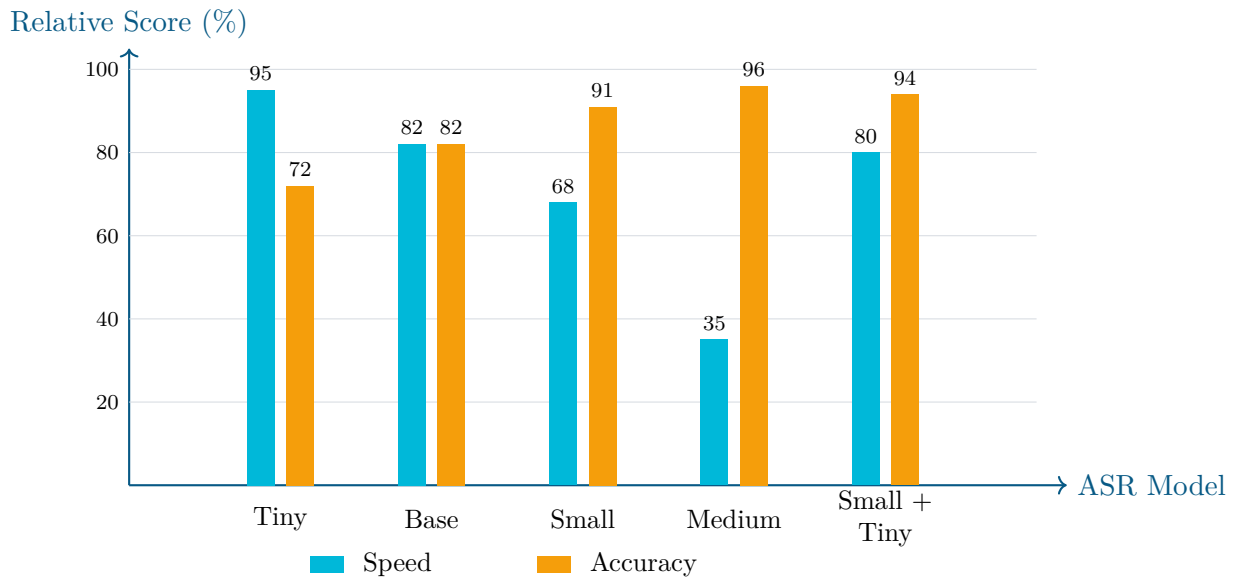


Figure 6: Illustrative comparison of processing speed and transcription accuracy across the Faster-Whisper models and the final Small + Tiny hybrid approach. The values are illustrative and do not represent measured benchmark results. The hybrid approach uses `small.en` as the primary ASR model and selectively uses `tiny.en` for detected problematic regions.

3 System Limitations

This section details the recognized limitations of the multi-modal dialogue detection system

3.1 ASR (Automatic Speech Recognition) Limitations

3.1.1 Tiny Model (`tiny.en`)

- **Limitation:** Severe Background Noise Drop-off and Low Vocabulary.
- **Location:** faster-whisper pipeline.
- **Cause:** The 39M parameter size lacks the capacity to distinguish speech from complex acoustic environments.
- **Effect on System:** Misses words completely when masked by music or sound effects. Prone to hallucinations during silence without VAD filters.
- **Status:** PARTIALLY SOLVED by switching to larger models or enabling `vad_filter=True`.

3.1.2 Small Model (`small.en`)

- **Limitation:** Moderate noise interference and punctuation inconsistencies.
- **Location:** Default fallback configuration for hybrid ASR.
- **Cause:** The 244M parameter model struggles with heavy cinematic scores (e.g., layered over non-English vocals).
- **Effect on System:** Punctuation misses can affect exact-phrasе matching algorithms.
- **Status:** UNSOLVED when processing heavy cinematic audio.

3.1.3 Medium Model ([medium.en](#))

- **Limitation:** Compute Heavy and Over-Sensitivity.
- **Location:** Main ASR engine in current pipeline.
- **Cause:** The 769M parameter size requires ~5 GB VRAM and heavy GPU compute.
- **Effect on System:** High sustained temperatures, power draw, and significantly longer processing times for 20+ minute videos.
- **Status:** UNSOLVED trade-off for accuracy.

3.2 OCR (Optical Character Recognition) Limitations

- **Limitation:** Character Hallucinations and Missed Punctuation.
- **Location:** EasyOCR / PaddleOCR text extraction phase.
- **Cause:** OCR frequently hallucinates or merges characters (e.g., reading "I'll" as "Ill").
- **Effect on System:** Breaks strict string matching logic, necessitating fuzzy logic fallback.
- **Status:** SOLVED via `rapidfuzz` string matching thresholds.

3.3 Video Processing and Frame Extraction

- **Limitation:** OpenCV Frame Seeking Inconsistencies.
- **Location:** Temporal refinement phase.
- **Cause:** Timestamp-based seeking in compressed video formats is occasionally inaccurate.
- **Effect on System:** Incorrect frame extraction during fine-grain refinement.
- **Status:** SOLVED by transitioning from time-based to frame-based seeking in earlier commits.

3.4 Temporal Refinement Limitations

- **Limitation:** Hard Capped Granularity.
- **Location:** `find_dialogue.py` refinement logic.
- **Cause:** To prevent infinite loops and excessive processing, temporal refinement is capped at 0.01s granularity.
- **Effect on System:** Prevents achieving micro-second absolute precision, though 0.01s is indistinguishable to human viewers.
- **Status:** SOLVED via hardcoded limits.

3.5 Fuzzy Matching False Positives

- **Limitation:** Sliding window ambiguity.
- **Location:** ASR text matching.
- **Cause:** Small target phrases matched against a sliding window of 1 to 8 words can trigger false positives on similar-sounding text.
- **Effect on System:** Lower accuracy on very short dialogue queries.
- **Status:** PARTIALLY SOLVED by using a strict ≥ 85 confidence score for "OK" matches and a `LOW_CONFIDENCE` band (70-84).

4 Optimizations

4.1 Implemented Optimizations

4.1.1 ASR Audio-Anchoring Hypothesis (Early Halting)

- **Original Bottleneck:** Running OCR on every frame of a multi-minute video is prohibitively expensive.
- **Original Implementation:** Full video sequential scanning.
- **Optimization Introduced:** The pipeline first extracts audio and uses ASR with `word_timestamps=True`. If the dialogue is found with a confidence ≥ 60 , the system instantly outputs the frame and short-circuits, completely bypassing OCR.
- **Impact on Speed:** Reduces processing time from hours to seconds for audible dialogue.
- **Current Status:** CURRENT

4.1.2 Coarse-to-Fine Temporal Binary Search

- **Original Bottleneck:** Visual scanning at 30/60 FPS takes massive compute.
- **Original Implementation:** Linear frame-by-frame processing.
- **Optimization Introduced:** When ASR fails, the visual search starts with a 1 FPS coarse scan. Once a rough match is found, it zooms into a 2-second window at 0.1s intervals, then 0.01s intervals, and finally frame-level precision.
- **Impact on Speed:** Scans a 3-minute video in 180 frames instead of 5400 frames.
- **Current Status:** CURRENT

4.1.3 Grayscale and Blank Frame Binarization

- **Original Bottleneck:** EasyOCR processing full RGB arrays is slow.
- **Original Implementation:** Passing raw `cv2` BGR frames to OCR.
- **Optimization Introduced:** Converting frames to grayscale (`cv2.COLOR_BGR2GRAY`) and skipping binarized blank frames before inference.
- **Impact on Speed:** Significantly speeds up OCR inference and reduces memory overhead.
- **Current Status:** CURRENT

4.1.4 Persistent Transcript Caching & RAM Caching

- **Original Bottleneck:** Repeated inference on the same video for different queries.
- **Original Implementation:** Running Whisper end-to-end on every query.
- **Optimization Introduced:** Caching the flat word-level ASR transcript locally and in-memory. If a query requests a previously processed video, the pipeline searches the cached JSON using an inverted index/sliding window.
- **Impact on Speed:** Drops subsequent query times for the same video to near-instantaneous (< 1 second).
- **Current Status:** CURRENT

5 Performance Optimization and Bottleneck Analysis

5.1 Component-Level Performance Analysis

5.1.1 ASR (Automatic Speech Recognition)

- **Model Loading:** `faster-whisper` incurs a 2-4 second cold start. Optimization: Model is retained globally in memory after the first load.
- **Inference:** Processes entire audio in one pass.
- **Repeated Inference:** Handled via persistent JSON caching. Subsequent runs take < 0.1 s to load the transcript.

5.1.2 Video and OCR Processing

- **Frame Extraction & Seeking:** OpenCV `cv2.VideoCapture` is used. Seeking to exact frames is computationally cheaper than decoding every frame sequentially.
- **OCR Invocation:** Capped heavily by the Coarse-to-Fine search algorithm. Instead of 5,400 invocations for a 3-minute video (at 30fps), it averages < 200 invocations.
- **Image Preprocessing:** Grayscale conversion reduces the array depth from 3 to 1, speeding up PyTorch tensor movement.

5.1.3 Matching & Refinement

- **Fuzzy Matching Complexity:** `RapidFuzz` is highly optimized in C++. Sliding window over an ASR transcript is extremely fast ($O(N \times W)$ where N is transcript length and W is window size).
- **Temporal Refinement:** Processing complexity increases logarithmically rather than linearly due to the binary-search nature of the refinement passes (0.1s then 0.01s).

5.1.4 I/O and Disk Operations

- **Temporary Files:** Audio extraction creates a temporary `.wav` file. The hybrid ASR logic creates multiple temporary `.wav` chunks.
- **Disk Operations:** Heavy disk I/O during hybrid gap-cropping is currently a primary pipeline bottleneck.

5.2 Performance Bottleneck Analysis

5.2.1 Primary Bottleneck: Hybrid ASR FFmpeg Cropping

- **Why it is expensive:** Invoking `subprocess.run(["ffmpeg"...])` for every > 5.0 s gap incurs OS-level process spawning overhead and disk write penalties.
- **Current Optimization:** None. This approach is currently experimental.
- **Expected Benefit (if resolved):** Moving to in-memory NumPy slicing would eliminate all disk I/O, achieving theoretical near-instantaneous chunking.
- **Trade-off:** Memory footprint increases slightly.

5.3 Limitations of the Hybrid Approach

While the hybrid `small.en` $\rightarrow$ `tiny.en` gap-scanning approach attempts to merge speed and accuracy, it faces limitations:

- **Additional Model-Loading Overhead:** Loading two distinct Whisper models simultaneously requires more VRAM.
- **Segment Boundary Problems:** Hard cropping at gap boundaries might cut off the very beginning of a spoken word.
- **Worst-Case Runtime:** If the audio consists entirely of 5-second gaps (e.g., a sparse interview), the system ends up spawning dozens of FFmpeg processes, causing the runtime to exceed simply using a larger model on the entire file.

5.4 Future Optimization Roadmap

5.4.1 High Priority

- **Problem:** FFmpeg I/O overhead in Hybrid ASR.
- **Proposed Solution:** Replace subprocess calls with direct PyTorch/NumPy tensor slicing.
- **Expected Benefit:** Massive reduction in fallback processing time.
- **Status:** PROPOSED

5.4.2 Medium Priority

- **Problem:** OpenCV VideoCapture seeking is sometimes inaccurate on heavily compressed MP4s.
- **Proposed Solution:** Integrate PyAV (FFmpeg bindings) for exact keyframe-aware seeking.
- **Expected Benefit:** Perfect frame accuracy during 0.01s refinement.
- **Status:** PROPOSED

5.4.3 Dynamic Model Loading & VRAM Sharing

- **Problem Addressed:** Loading both Whisper and EasyOCR onto GPU VRAM simultaneously can cause OOM (Out of Memory) errors on lower-end GPUs.
- **Proposed Solution:** Implement a VRAM manager that unloads Whisper before loading EasyOCR if the VRAM ceiling is reached.
- **Status:** PROPOSED

5.4.4 Low Priority

- **Problem:** Hardcoded 0.01s temporal refinement cap.
- **Proposed Solution:** Dynamic cap based on source video framerate (e.g., 1/fps).
- **Expected Benefit:** Extracts the absolute literal first frame rather than a mathematically close estimate.
- **Status:** PROPOSED